

Sorting by Swapping Adjacent Elements Is Bad, Really.

BSc Computer Science and Artificial Intelligence

School of Informatics and Engineering, University of Sussex

Sajed Albarah

Project Supervisor: Dr Vincent van Oostrom

August 2026

A dissertation submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science.

Statement of Originality

This report is submitted as part of the requirements for the degree of BSc Computer Science and Artificial Intelligence at the University of Sussex. It is the product of my own work except where stated otherwise in the text. All sources have been acknowledged appropriately. I give permission for this report to be copied and distributed for academic purposes, provided that the source is acknowledged.

Sajed Albarah
Date: August 2026

Abstract

Sorting by repeatedly exchanging adjacent elements is easy to implement, but the restriction to adjacent exchanges imposes an unavoidable cost. This project formalises that cost in Isabelle/HOL. Two complementary developments are given. The first is an elementary proof intended to introduce the reader to Isabelle syntax. It defines the inversion count of a list, proves that swapping an adjacent inversion decreases this count by exactly one, and concludes that any recorded inversion-swap reduction ending in a sorted list has length equal to the initial inversion count. Since a list of length n has at most $\binom{n}{2}$ inversions, this gives an exact quadratic worst case for this class of reductions.

The second development formalises the abstract rewriting account of random descent from van Oostrom's Example 7. A generic abstract reduction system is defined with finite and infinite reductions, conversions, normal forms and strategies. The development proves the general implications of ordered local commutation, global strategy comparison, ordered weak Church-Rosser, random descent, etc. These results are then instantiated with lists. The ambient system permits any adjacent exchange while a list is unsorted, whereas the specific strategy exchanges only adjacent inversions. A complete analysis of disjoint and overlapping local peaks establishes random descent for the inversion strategy and proves that it is no longer than arbitrary adjacent swapping. Finally, the descending list $\text{rev}[0.. < n]$ is shown to require at least $\binom{n}{2}$ adjacent exchanges in every terminating sorting reduction. The result is a machine-checked $\Omega(n^2)$ worst-case lower bound on swap complexity.

Acknowledgements

I would like to thank my supervisor, Dr Vincent van Oostrom, for introducing me to the rewriting perspective behind this project and for his advice throughout its development.

Contents

Statement of Originality	i
Abstract	ii
Acknowledgements	iii
1 Introduction	1
1.1 Problem Area	1
1.2 Motivation	1
1.3 Aims and Objectives	2
1.4 Report Structure	2
1.5 Ethical Considerations	2
1.6 Reproducibility and Research Integrity	3
2 Background and Related Work	4
2.1 Inversions and Adjacent-Swap Sorting	4
2.2 Abstract Reduction Systems	4
2.3 Strategies and Random Descent	5
2.4 Proof Assistants and Isabelle/HOL	5
3 Formalisation Design	7
3.1 Project Scope	7
3.2 Modelling Decisions	7
3.3 High-Level Architecture of the Isabelle Development	8
3.4 Definitions and Theorem Dependencies	8
4 Direct Proof	10
4.1 Formalisation	10

4.1.1	The Adjacent-Swap Reduction	10
4.1.2	Inversions and the Number of Sorting Steps	11
5	General Rewriting Framework	13
5.1	Abstract Reduction Systems	13
5.2	Reductions and Normal Forms	13
5.3	Strategies	14
5.4	Strategy Comparison Results	15
6	Formalisation of Adjacent-Swap Sorting	17
6.1	Lists, Orders, and Sortedness	17
6.2	Adjacent Inversions and Swap Steps	17
6.3	Inversion Count	18
6.4	Termination	19
6.5	Strategy-Independent Reduction Length	19
6.6	Quadratic Worst-Case Complexity	20
7	Evaluation and Discussion	22
7.1	Evaluation Against Objectives	22
7.2	Verification Evidence	23
7.3	Representative Examples	23
7.4	Discussion of the Results	24
8	Limitations and Future Work	26
8.1	Limitations	26
8.2	Future Work	26
9	Conclusion and Reflection	27

Chapter 1

Introduction

1.1 Problem Area

Sorting is one of the standard problems used to explain both algorithm design and complexity analysis. The input is a finite sequence and the output is a permutation of the same elements arranged in nondecreasing order. Many familiar algorithms satisfy this specification, but they do so using different primitive operations. This project studies the consequences of choosing one particularly restrictive primitive operation: exchanging two adjacent list elements.

An adjacent exchange can move an element by only one position, which acts as a ‘speed limit’ on how quickly elements of a list can move around. On a descending list, every pair of positions is in the wrong relative order, so a large amount of local movement is unavoidable. This observation suggests a quadratic lower bound, but the project is concerned with a stronger account. It asks how the choice of an adjacent pair affects the number of exchanges, how the answer can be stated as a property of a reduction system, and how the argument can be checked by a proof assistant.

The cost model used in the main theorem counts swaps. It does not count array accesses, comparisons, loop overhead or the cost of deciding which pair to exchange. This is still a useful lower-bound model: an implementation that performs m swaps must take at least the time required to perform those m primitive operations. The final theorem therefore gives an $\Omega(n^2)$ worst-case time lower bound for any sorting procedure represented by the formal model, provided an adjacent exchange has non-zero constant cost.

1.2 Motivation

The paper *Random Descent* introduces local diagram conditions that can be used to compare reduction strategies globally [7]. Its Example 7 applies the method to sorting by adjacent swaps. The example is mathematically persuasive and intuitive, but much of the necessary structure is implicit: finite and infinite reductions, the meaning of a strategy, normal forms, the ordering on reduction lengths, and the critical-pair arguments all have to be made precise before a proof assistant can check it.

This makes the example a useful formalisation project. The mathematical result is concrete

enough to explain using lists, yet its proof requires reusable abstract machinery. It also illustrates a central benefit of interactive theorem proving. A paper proof can say that the remaining critical pairs are easy or rely on (easy) geometric intuitions; a formal proof must state exactly which indices overlap, how the list is decomposed, what the joining reductions are, and why their lengths satisfy the required inequality.

1.3 Aims and Objectives

The final scope of the project is expressed by the following objectives.

1. Formalise adjacent inversion swaps and prove directly that the number of swaps in a completed reduction is the initial inversion count.
2. Define a reusable abstract reduction system in Isabelle/HOL, including finite and infinite reductions, conversions, normal forms and strategies.
3. Formalise the strategy-comparison results needed by [7], in particular $\text{OLCOM}(S, I) \Rightarrow \text{better}(S, I)$ and $\text{OWCR}(S) \Rightarrow \text{RD}(S)$.
4. Instantiate this framework with the ambient adjacent-swap system and the inversion-only strategy, discharging all local peaks.
5. Prove an exact quadratic lower bound on the descending family of inputs and state carefully what this implies about adjacent-swap sorting.

The proposal also discussed formalising non-adjacent algorithms and swaps of a bounded distance. Those are useful extensions, but they are not required for the theorem proved here and were removed from the final formalisation due to time constraints. This is discussed more in Chapter 9.

1.4 Report Structure

Chapter 2 discusses professional practice, ethics and reproducibility. Chapter 3 introduces inversions, sorting complexity, abstract reduction systems, random descent and Isabelle/HOL. Chapter 4 explains the design and dependency structure of the formalisation. Chapter 5 presents the direct proof and uses it to introduce the main Isabelle constructs. Chapter 6 describes the generic rewriting and strategy framework. Chapter 7 gives the list instantiation and the final lower bound. Chapter 8 evaluates the result and gives representative examples. Chapter 9 records limitations and possible extensions. Chapter 10 concludes with a reflection on the proof process.

1.5 Ethical Considerations

The project uses no human participants, personal data, private datasets or deployed decision-making system. It does not create a realistic risk to public health, privacy or security. Consequently, no ethical review is required.

There are still general ethical considerations. Formal verification can encourage unjustified confidence when the formal model is narrower than the system described informally. Here the model counts adjacent swaps and assumes that list elements have a linear order. It does not establish the wall-clock performance of source code, and it does not verify memory use or comparison counts. These boundaries are stated explicitly rather than hidden behind the phrase “formally verified.”

The source material and external definitions are cited, and No personal or confidential information is included in the theories.

1.6 Reproducibility and Research Integrity

The authoritative output consists of plaintext Isabelle theory files. The proof dependency chain is explicit, and all the files compile with no errors . The source theories are the verification evidence and can be processed independently of the prose. The report, however, only shows selected definitions, lemmas, theorems, and proofs. The full Isabelle files cannot be inferred from the text alone, so they must be checked independently.

Automation such as `simp`, `auto`, `blast` and arithmetic methods were used for routine goals. These methods do not replace the logical checker, but they construct proof results that Isabelle’s kernel checks in the same way structured Isar proofs [6]. The larger arguments remain explicit in the theory. regardless, the Isabelle formalisation should run reliably when tested independently.

Chapter 2

Background and Related Work

2.1 Inversions and Adjacent-Swap Sorting

Let $x = [a_0, a_1, \dots, a_{n-1}]$ be a finite list over a linearly ordered type. An inversion is a pair of indices (i, j) satisfying

$$0 \leq i < j < n \quad \text{and} \quad a_j < a_i.$$

The inversion count $\text{inv}(x)$ is the number of such pairs. A list is sorted in nondecreasing order exactly when $\text{inv}(x) = 0$. Duplicates cause no problems because the inequality is strict.

If two adjacent elements $a_i > a_{i+1}$ are swapped, their inversion disappears. Every earlier element remains before both values, every later element remains after both values, and all pairs not involving these two positions keep their relative order. Hence, total inversion count falls by one. Repeating this argument shows that a sequence of adjacent inversion exchanges which reaches a sorted list has length $\text{inv}(x)$, regardless of which available inversion is chosen at each step.

There are $\binom{n}{2}$ pairs of distinct positions, so

$$0 \leq \text{inv}(x) \leq \binom{n}{2} = \frac{n(n-1)}{2}.$$

When the elements are distinct and descending, every pair is inverted and the upper bound is attained. This elementary argument is the basis of Chapter 5.

2.2 Abstract Reduction Systems

An abstract reduction system (ARS) consists of a set of objects and a directed step relation, it's studied in the field of term rewriting and is often the first introduction to it. The notation $a \rightarrow b$ means that one step transforms a into b . A finite reduction is a sequence

$$a = a_0 \rightarrow a_1 \rightarrow \dots \rightarrow a_k = b,$$

with length k . A normal form has no outgoing step. Infinite reductions are also important because a strategy comparison shouldn't tacitly assume that every sequence terminates. Introductions to these notions and term rewriting rewriting are given by Baader and Nipkow [1], there are many other reliable sources as well.

The sorting instance uses finite lists as objects. A step contains a source list and an index, and its target is obtained by exchanging the values at that index and its successor. Keeping the step itself, rather than representing only an unlabelled pair of objects, makes the location of a critical pair available inside Isabelle. Imagine each a_i as a permutation of the same list and arrows between them representing the steps of sorting.

2.3 Strategies and Random Descent

In this project a strategy is a sub-ARS using the same objects and having the same normal forms as the ambient system; what this means that a strategy keeps the same objects and the same final a_k without introducing any new normal forms, but restricts the arrows.

The relation $\text{better}(S, I)$ states that every maximal S reduction is no longer than every maximal I reduction from the same object. Reduction lengths use extended natural numbers, so infinite paths have length ∞ and can be compared with finite paths in one definition.

Ordered local commutation, $\text{OLCOM}(S, I)$, is a local diagram condition. From a peak containing one S step and one I step, it requires either an appropriate infinite continuation or finite joining reductions satisfying a length inequality. Theorem 2 of [7] turns this local condition into the global comparison $\text{better}(S, I)$. Ordered weak Church-Rosser is the self-comparison $\text{OWCR}(S) = \text{OLCOM}(S, S)$. Theorem 3 gives

$$\text{OWCR}(S) \implies \text{RD}(S).$$

Random descent says that once a normal form is connected to the source by an appropriate conversion, every maximal strategy reduction is finite, reaches that normal form, and has the same length. This is how we prove that all adjacent swap sorting algorithms have the same fundamental limitation. All of this should make more sense to the unfamiliar reader later on.

2.4 Proof Assistants and Isabelle/HOL

Isabelle is a generic proof-assistant, and Isabelle/HOL is widely used for higher-order logic. HOL provides typed functional programming constructs, and mathematical libraries, while Isar provides a structured proof language [6, 4] that makes proofs much more readable and more similar to mathematical prose.

An Isabelle file is called a theory and it has the following structure:

```
theory Example
  imports Main
begin
```

```
lemma append_Nil: "xs @ [] = xs"
  by (induction xs) simp_all

end
```

The text between quotation marks is a HOL proposition, and is referred to as the inner “syntax”. The command `lemma` introduces it, `induction xs` selects list induction, and `simp_all` uses rewrite rules to solve the resulting cases. The official Isabelle documentation treats theories, recursive functions, inductive definitions in this functional style [4]. The direct proof in Chapter 5 applies the same constructs to the project.

Chapter 3

Formalisation Design

3.1 Project Scope

The formalisation has two parts. The direct parts considers lists of natural numbers and only permits swaps of adjacent inversions. It proves an exact equation for every supplied reduction ending in a sorted list and then derives a quadratic upper bound. This is deliberately and introduces the material before getting deep into rewriting.

The abstract part separates a generic rewriting framework from its sorting instance. It handles both finite and infinite maximal reductions, defines strategy comparison, proves the required general theorems, and finally models arbitrary adjacent swaps. Its final theorem is a worst-case lower bound: every terminating ambient reduction which sorts the descending list has length at least $\binom{n}{2}$. The model does not attempt to verify executable bubble sort or any other sorting algorithm.

3.2 Modelling Decisions

Several design choices were important.

Steps are first-class values.

A list ARS step stores its source and swap index. This allows two different swap positions with the same source to be distinguished and makes index-based criticalpair analysis possible.

Finite paths record successive targets.

A value `finite_reduction_on S a rs b` uses the list `rs` to store one target per step. Hence `length rs` is the cost without a separate counting function.

Infinite paths are streams.

Coinductive streams represent infinite reductions (which are defined coinductively in the paper anyway), and the datatype `reduction` combines finite and infinite cases. Its length lies in the extended naturals, where every finite value is below infinity.

The main list development is agnostic to list type.

Most of `Main_Result` works for lists over any Isabelle type in the `linorder` class,

which is what you need for a sorting algorithm to work. Natural numbers appear only in the concrete family $\text{rev}[0.. < n]$ used to state the final bound.

The direct and abstract parts of the formalisation each define an inversion-count function. This is some duplication, but it keeps theories independent and lets it use a much smaller set of definitions. The theories in this project, are not intended to be library theories, and aren't as reusable.

3.3 High-Level Architecture of the Isabelle Development

The final proof development contains 86 named lemmas, theorems and corollaries. The theories have the following roles.

Theory	Role
<code>Adjacent_Swap_Sorting</code>	Independent direct proof using inversion count and an explicit step counter on lists with numbers.
<code>ARS</code>	Objects, labelled steps, conversions, finite and infinite reductions, normal forms and maximal reductions.
<code>Strategy_comparison</code>	Definitions of strategies, minimal and maximal behaviour, better , olcom and bounded .
<code>Strategy_theorems</code>	Generic reduction lemmas and proofs of $\text{OLCOM} \Rightarrow \text{better}$ and $\text{OWCR} \Rightarrow \text{RD}$, with consequences of random descent.
<code>Main_Result</code>	Adjacent list swaps, critical-pair analyses, inversion potential, existence and cost of maximal reductions, and the final quadratic lower bound.

The import chain is linear for the abstract development with each file building on the one before:

$$\text{ARS} \longrightarrow \text{Strategy_comparison} \longrightarrow \text{Strategy_theorems} \longrightarrow \text{Main_Result}.$$

`Adjacent_Swap_Sorting` is intentionally independent. It can be read first without learning the generic `ARS` representation. A separate document wrapper can import the theories and use Isabelle's antiquotations to print checked theorem statements in the report.

3.4 Definitions and Theorem Dependencies

The direct proof has a short dependency chain:

```

inv_swap_middle  $\Rightarrow$  adj_swap_decreases_inv  $\Rightarrow$  sorting_steps_inv
 $\Rightarrow$  steps_to_sort,
inv_le_choose_two  $\Rightarrow$  inv_quadratic_bound,
steps_to_sort + inv_quadratic_bound  $\Rightarrow$  adjacent_swap_sorting_complexity.

```

The abstract proof divides into a generic and an instantiated part. In the generic part, local commutation is first extended from one peak to finite diagrams. This establishes **bounded**; boundedness then compares maximal reductions and proves **better**. In the self case, boundedness is turned into **self_bounded**, which is strong enough to prove random descent. Thus the two main chains are

$$\text{OLCOM}(S, I) \Rightarrow \text{bounded}(S, I) \Rightarrow \text{better}(S, I)$$

and

$$\text{OWCR}(S) \Rightarrow \text{self_bounded}(S) \Rightarrow \text{RD}(S).$$

In the list instance, swap algebra and index decompositions support the local peak lemmas. These prove **inversion_owcr** and **inversion_olcom_adjacent**, from which the generic theorems yield **inversion_random_descent** and **inversion_better_adjacent**. Separately, the inversion count proves termination and prices a maximal inversion reduction on the descending input. The global comparison finally transfers that price into the lower bound for every ambient reduction to a sorted list.

Chapter 4

Direct Proof

4.1 Formalisation

The direct theory is presented before the general rewriting framework. Its purpose is both mathematical and explanatory. It proves the inversion-count argument with only recursive functions, inductive predicates, list induction and arithmetic. These are close to ordinary functional programming, so the reader can learn the syntax while following an elementary proof.

The theory imports Isabelle's main HOL library and the binomial-coefficient library. It specialises values to natural numbers:

```
theory Adjacent_Swap_Sorting
  imports Main "HOL.Binomial"
begin
```

The quotation marks surrounding types and propositions separate Isabelle's outer command language from HOL's inner syntax. For example, `nat list` is the HOL type of lists of natural numbers, while `lemma` and `proof` are theory commands from the outer syntax.

4.1.1 The Adjacent-Swap Reduction

The inversion count is defined by recursion on the list as you would expect in functional programming:

```
fun inv :: "nat list ⇒ nat" where
  "inv [] = 0"
| "inv (x # xs) =
    inv xs + length (filter (λy. y < x) xs)"
```

The empty list has no inversions. For `x # xs`, the recursive call counts inversions completely inside `xs`; the filtered list counts exactly those later values smaller than `x`.

Isabelle's existing predicate `sorted` states that a list is in nondecreasing order. The first lemma connects the custom function to this library notion:

```

lemma inv_zero_iff_sorted:
  "inv xs = 0  $\longleftrightarrow$  sorted xs"
  by (induction xs) (auto simp: filter_empty_conv)

```

This is a compact proof by list induction. In the nonempty case, `filter_empty_conv` turns the absence of a smaller later element into an order condition on the head, while the induction hypothesis deals with the tail.

One permitted sorting step is an inductive relation. The source is decomposed into a prefix, two adjacent values and a suffix:

```

inductive adj_swap :: "nat list  $\Rightarrow$  nat list  $\Rightarrow$  bool"
  (infix " $\rightsquigarrow$ " 50)
where
  swap:
    "y < x  $\implies$ 
    pre @ (x # y # suf)  $\rightsquigarrow$  pre @ (y # x # suf)"

```

Only an inverted pair may be exchanged. The variables `pre` and `suf` allow the rule to apply at any valid position.

A second inductive predicate adds an explicit step count:

```

inductive sorting_steps ::
  "nat list  $\Rightarrow$  nat  $\Rightarrow$  nat list  $\Rightarrow$  bool"
where
  zero: "sorting_steps xs 0 xs"
| step: "xs  $\rightsquigarrow$  ys  $\implies$ 
  sorting_steps ys n zs  $\implies$ 
  sorting_steps xs (Suc n) zs"

```

The `zero` rule is the empty reduction. The `step` rule prefixes one exchange and increments the recorded natural number.

4.1.2 Inversions and the Number of Sorting Steps

The key local lemma is that one permitted exchange removes exactly one inversion:

```

lemma adj_swap_decreases_inv:
  assumes "xs  $\rightsquigarrow$  ys"
  shows "inv xs = Suc (inv ys)"

```

Its proof first establishes `inv_swap_middle` for an explicit prefix. Induction on the prefix moves the argument to the required position. The helper `length_filter_swap` shows that exchanging two elements does not change the number satisfying any unrelated predicate. Together these facts formalise the paper argument that the exchanged pair is the only pair whose inversion status changes overall.

Induction over a `sorting_steps` derivation then sums the one step equation:

```

lemma sorting_steps_inv:
  assumes "sorting_steps xs n ys"

```

```
shows "inv xs = n + inv ys"
```

If the endpoint is sorted, $\text{inv } \text{ys} = 0$. The exact step result is therefore:

```
theorem steps_to_sort:
  assumes steps: "sorting_steps xs n ys"
    and final: "sorted ys"
  shows "n = inv xs"
```

The theorem is conditional on a supplied finite reduction and a sorted endpoint. It says that every such inversion-only path from the same list has the same length. It does not say that paths which are allowed to exchange ordered pairs have the same length.

The quadratic bound is proved independently by induction on the input list:

```
lemma inv_le_choose_two:
  "inv xs ≤ (length xs choose 2)"
```

Adding a new head can create at most $\text{length } \text{xs}$ new inversions. The induction hypothesis bounds the tail by $\binom{|xs|}{2}$, and the binomial identity

$$\binom{k+1}{2} = k + \binom{k}{2}$$

closes the induction. Rewriting the binomial coefficient yields:

$$\text{inv}(xs) \leq \frac{|xs|(|xs| - 1)}{2},$$

where Isabelle's theorem is stated using natural number multiplication, subtraction and division. The final direct theorem combines the exact and upper bound results:

```
theorem adjacent_swap_sorting_complexity:
  assumes steps: "sorting_steps xs n ys"
    and final: "sorted ys"
  shows "n = inv xs"
    and "n ≤ length xs * (length xs - 1) div 2"
```

This theorem already explains why bubble style and insertion style choices perform the same number of actual inversion exchanges on a fixed input. The later development gives a different result because it compares this with the larger class of all adjacent exchanges and obtains a lower bound for the larger class.

Chapter 5

General Rewriting Framework

5.1 Abstract Reduction Systems

The generic framework is contained in the theories `ARS`, `Strategy_comparison` and `Strategy_theorems`. Its base is an Isabelle locale:

```
locale ars =  
  fixes Obj  :: "'o set"  
    and Step :: "'s set"  
    and src  :: "'s  $\Rightarrow$  'o"  
    and tgt  :: "'s  $\Rightarrow$  'o"  
  assumes src_closed: "e  $\in$  Step  $\implies$  src e  $\in$  Obj"  
    and tgt_closed: "e  $\in$  Step  $\implies$  tgt e  $\in$  Obj"
```

A locale fixes a context and assumptions which are available to all definitions and lemmas declared inside it. The types of objects and steps are independent. A set `S` of selected step values induces the object-level relation

```
definition rstep_on where  
  "rstep_on S a b  $\longleftrightarrow$   
    ( $\exists e \in S. \text{src } e = a \wedge \text{tgt } e = b$ )"
```

Using labelled steps is slightly more elaborate than fixing a binary relation, but it allows a strategy to select particular step occurrences. This is useful for lists, where an edge is identified by its source and index.

The locale assumptions ensure that every ambient edge begins and ends in `Obj`. Strategies later inherit this closure by being subsets of `Step`. The sorting interpretation chooses all linearly ordered lists as objects, so object closure is immediate.

5.2 Reductions and Normal Forms

Finite reductions are represented inductively:

```
inductive finite_reduction_on ::  
  "'s set  $\Rightarrow$  'o  $\Rightarrow$  'o list  $\Rightarrow$  'o  $\Rightarrow$  bool"
```

```

for S
where
  fr_nil: "finite_reduction_on S a [] a"
| fr_cons:
  "rstep_on S a b ==>
   finite_reduction_on S b rs c ==>
   finite_reduction_on S a (b # rs) c"

```

If $\mathbf{rs}$ contains k successive target objects, the reduction has length k . Basic lemmas prove the expected empty-list characterisation, decomposition of a nonempty path, concatenation, monotonicity under a larger step set, and closure of endpoints.

Infinite reductions use a coinductive predicate over streams. Coinduction is appropriate because an infinite object is observed one step at a time: the head must be a valid target and the tail must again be an infinite reduction. This also follows how infinite reductions are defined in [7]. The datatype

```

datatype 'o reduction =
  FinRed "'o list" 'o
| InfRed "'o stream"

```

places finite and infinite cases under one type. The function `reduction_length` returns an extended natural number: a finite path has the length of its target list and an infinite path has length infinity.

A normal form relative to S has no outgoing S edge. A maximal reduction is either infinite or finite and ending in such a normal form. This definition is important. “Maximal” describes a path which cannot be extended, not necessarily a longest finite path.

The framework also defines finite conversions using explicit forward and backward step constructors. Their signed distance satisfies append and inverse lemmas:

$$\begin{aligned}
d(cs@ds) &= d(cs) + d(ds), \\
d(cs^{-1}) &= -d(cs).
\end{aligned}$$

Every finite reduction can be viewed as a forward-only conversion whose signed distance and ordinary length agree. These algebraic facts are later used when random descent compares a maximal reduction with a conversion to normal form.

5.3 Strategies

The definition is:

```

definition strategy where
  "strategy S <=>
   sub_ars Obj S ^
   (∀a∈Obj.
    normal_form_on S a <=> normal_form a)"

```

Thus a strategy uses a subset of ambient steps but preserves the ambient normal forms.

The development formalises several properties from [7]. A normalising strategy has only finite maximal reductions whenever an ambient normal form is reachable. A minimal strategy uses no more steps than an ambient reduction to the same normal form. The dual notions `perpetual_strategy` and `maximal_strategy` concern infinite behaviour and longest reductions. The main list result only requires the more general two-strategy comparison `better`:

```

definition better where
  "better S I  $\longleftrightarrow$ 
    strategy S  $\wedge$  strategy I  $\wedge$ 
    ( $\forall a \in \text{Obj}. \forall R T.$ 
      maximal_reduction_from_on S a R  $\longrightarrow$ 
      maximal_reduction_from_on I a T  $\longrightarrow$ 
      reduction_length R  $\leq$  reduction_length T)"

```

If the right-hand strategy has an infinite maximal path, the inequality is automatic. If it has a finite maximal path, `better` rules out an infinite path on the left and compares the two natural lengths. This explains why the generic theory has infinite reductions even though the final sortinglower bound assumes a finite sorting path.

5.4 Strategy Comparison Results

Ordered local commutation is defined as a local condition on one step from each strategy. In the finite branch, the two targets must be joinable by reductions whose lengths satisfy the orientation required for comparison. The alternative branch permits an infinite continuation where this is the correct global outcome. The auxiliary property `bounded S I` lifts this shape from one peak to arbitrary finite reductions.

The most substantial generic lemma, `olcom_finite_diagram`, performs this lifting. Its proof inducts through the relevant reduction and composes local joining diagrams and preserving the length inequality. This gives

```

theorem olcom_imp_bounded:
  assumes "olcom S I"
  shows "bounded S I"

```

Boundedness then proves the global comparison. When both maximal reductions are finite, its length inequality is immediate after discarding the nonnegative joining suffix. When the left path is infinite but the right path reaches a normal form, arbitrarily long finite prefixes contradict the bound. The two steps are packaged as Theorem 2:

```

theorem olcom_imp_better:
  assumes "olcom S I"
  shows "better S I"

```

For random descent, `owcr S` is defined as `olcom S S`. The self instance first produces `self_bounded`. This states that a conversion to a normal form can be replaced by a forward reduction whose length is no greater than the conversion's signed distance. The proof of `self_bounded_imp_random_descent` uses a conversion and its inverse to obtain both upper and lower bounds on every finite maximal path. An infinite maximal path is

excluded by taking a finite prefix one step longer than the derived bound. The resulting formal version of Theorem 3 is:

```
theorem owcr_imp_random_descent:  
  assumes "owcr S"  
  shows "random_descent S"
```

Chapter 6

Formalisation of Adjacent-Swap Sorting

6.1 Lists, Orders, and Sortedness

The main instantiation works over `'a::linorder list`. The type-class constraint supplies a reflexive, antisymmetric, transitive and total order, but does not commit the development to natural numbers. Isabelle's library predicate `sorted` is used as the specification of a final list.

One adjacent exchange is implemented by a total recursive function:

```
fun swap_adj_at :: "nat  $\Rightarrow$  'a list  $\Rightarrow$  'a list" where
  "swap_adj_at 0 (x # y # xs) = y # x # xs"
| "swap_adj_at (Suc i) (x # xs) = x # swap_adj_at i xs"
| "swap_adj_at i xs = xs"
```

The last equation makes invalid indices harmless by returning the original list. Step membership separately requires `Suc i < length xs`, so every actual ARS edge performs a real exchange.

The elementary lemmas show that a swap preserves length and multiset, can be located inside a prefix and suffix, is an involution at a valid index, and commutes with a sufficiently distant swap. Length and multiset preservation establish that the operation produces a permutation of the input. The commuting lemma is used in later diagrams and proofs.

6.2 Adjacent Inversions and Swap Steps

A labelled list step stores a source list and index:

```
datatype 'a list_step = List_Step "'a list" nat
```

The source projection returns the stored list, the index projection returns its position, and the target projection applies `swap_adj_at`. Two sets of steps are then defined:

```

definition adjacent_steps where
  "adjacent_steps =
    {e. Suc (list_index e) < length (list_src e)
     ∧ ¬ sorted (list_src e)}"

definition inversion_steps where
  "inversion_steps =
    {e ∈ adjacent_steps.
     list_src e ! Suc (list_index e)
     < list_src e ! list_index e}"

```

The locale interpretation `list_ars` chooses all lists as objects and `adjacent_steps` as the ambient edge set. Theorems `adjacent_rstep_iff` and `inversion_rstep_iff` unfold a locale step into the concrete existence of an index, the order condition, and the resulting target.

The proof that both systems have sorted normal forms establishes that every unsorted linearly ordered list contains an adjacent inversion. Therefore an unsorted list has an outgoing inversion step, and hence also an ambient step. Conversely, the definition forbids any ambient step from a sorted source. The resulting simplification rules are

$$\begin{aligned} \text{normal_form}_{\text{adjacent}}(xs) &\iff \text{sorted}(xs), \\ \text{normal_form}_{\text{inversion}}(xs) &\iff \text{sorted}(xs). \end{aligned}$$

Together with `inversion_steps` $\subseteq$ `adjacent_steps`, these facts prove that the inversion subset is a strategy and that the ambient system is a strategy of itself.

6.3 Inversion Count

The generic list theory defines a polymorphic inversion count with the same recursion as the direct theory:

```

fun inversion_count :: "'a::linorder list ⇒ nat" where
  "inversion_count [] = 0"
| "inversion_count (x # xs) =
    inversion_count xs
    + length (filter (λy. y < x) xs)"

```

The lemma `inversion_count_zero_iff_sorted` connects the value zero with the already established normal forms. The main local cost theorem states that swapping a valid adjacent inversion decreases the count by exactly one:

```

lemma inversion_count_swap_adjacent_inversion:
  assumes "Suc i < length xs"
    and "xs ! Suc i < xs ! i"
  shows "Suc (inversion_count (swap_adj_at i xs)) =
    inversion_count xs"

```

Induction over a finite inversion reduction accumulates this equation. If `rs` leads from `xs` to `ys`, then

$$\text{inversion_count}(xs) = |rs| + \text{inversion_count}(ys).$$

This equation is useful twice. It proves termination because the count cannot decrease forever, and it calculates the exact length of any finite inversion reduction which ends sorted.

For the concrete descending list, the development proves

$$\text{inversion_count}(\text{rev}[0..<n]) = \binom{n}{2}.$$

The elements of $[0..<n]$ are distinct, and reversing them puts every pair in descending order. Stating the result with Isabelle's binomial coefficient avoids an unnecessary detour through integer-valued asymptotic notation.

6.4 Termination

The theorem `inversion_maximal_exists` constructs a finite maximal inversion reduction from every linearly ordered list. It uses well-founded induction on `inversion_count xs`, see [1] for more information on WFI.

If the count is zero, the list is sorted and the empty reduction is maximal. Otherwise the list is unsorted, so the adjacent-inversion lemma supplies an index. Swapping there strictly decreases the count. The induction hypothesis provides a maximal reduction from the target, and the proof prefixes the chosen step. This is a formal version of the usual termination argument.

Termination can also be read from the accumulated cost equation, but the explicit existence theorem is needed by the final comparison. Given an ambient sorting path, the proof must exhibit a maximal inversion path from the same source before `better` can compare their lengths.

6.5 Strategy-Independent Reduction Length

Local peaks inside the inversion strategy are checked. If both steps use the same index, their targets are equal. If their indices are disjoint, the swaps commute and the original inversions remain available on the opposite branches. If the indices overlap, they must be adjacent positions. The order assumptions force a three-element descending pattern, and the braid law gives the joining diagram. These cases are assembled into `inversion_local_commutation`, then into

```
lemma inversion_owcr:
  "list_ars.owcr inversion_steps"

theorem inversion_random_descent:
  "list_ars.random_descent inversion_steps"
```

The generic consequence of random descent is that every maximal inversion reduction from a fixed source has the same endpoint and length. Thus the choice between available inversions does not affect the number of exchanges.

Second, a mixed local analysis compares one inversion step with one arbitrary ambient step. If the ambient step is also an inversion, the self-commutation result is reused. Otherwise the proof splits into disjoint positions and the two orientations of an overlap. The formal lemmas construct the required joining reductions and prove the local length inequality. They yield

```
lemma inversion_olcom_adjacent:
  "list_ars.olcom inversion_steps adjacent_steps"

theorem inversion_better_adjacent:
  "list_ars.better inversion_steps adjacent_steps"
```

This second theorem states an inequality, not equality. A maximal inversion reduction is no longer than a maximal ambient adjacent reduction from the same source. The difference is essential because an ambient path can make unproductive exchanges.

6.6 Quadratic Worst-Case Complexity

Random descent and the inversion potential first price every maximal inversion reduction on the descending family:

```
theorem inversion_maximal_length_rev_upt:
  assumes
    "list_ars.maximal_reduction_from_on inversion_steps
      (rev [0.. $n$ ]) (FinRed rs ys)"
  shows "length rs =  $n$  choose 2"
```

The final theorem begins with an arbitrary finite ambient reduction from the same descending list and assumes that its endpoint is sorted. Sortedness makes the reduction maximal because sorted lists are ambient normal forms. The existence theorem supplies a finite maximal inversion reduction. The global comparison `inversion_better_adjacent` says the inversion path is no longer than the ambient path, while the theorem above fixes the inversion path at $\binom{n}{2}$. Combining the two facts proves:

```
theorem adjacent_swap_sorting_lower_bound:
  assumes reduction:
    "list_ars.finite_reduction_on
      adjacent_steps (rev [0.. $n$ ]) rs ys"
    and sorted: "sorted ys"
  shows " $n$  choose 2  $\leq$  length rs"
```

Since

$$\binom{n}{2} = \frac{n(n-1)}{2},$$

this is an exact quadratic lower bound on a family of length- n inputs. Any algorithm which always sorts using steps from `adjacent_steps` must, on this family, perform at least that many swaps. Its worst-case swap complexity is therefore $\Omega(n^2)$.

For algorithms which exchange only adjacent inversions, the direct upper bound and the descending lower bound match, giving $\Theta(n^2)$ worst-case swap complexity. For the larger ambient class there is no common quadratic upper bound: an algorithm may take avoidable detours or fail to terminate. The proved common statement for that class is the lower bound.

Chapter 7

Evaluation and Discussion

7.1 Evaluation Against Objectives

Table 7.1 evaluates the completed work against the final objectives from Section 1.3. The assessment separates formal results from interpretation so that an informal asymptotic claim is not counted as a machine-checked theorem unless its exact finite form appears in the theories.

Table 7.1: Evaluation against project objectives.

Objective	Status	Evidence
Direct inversion-count proof	Achieved	<code>steps_to_sort</code> proves the exact count and the final direct theorem proves the quadratic upper bound.
Generic ARS and strategies	Achieved	<code>ARS</code> , <code>Strategy_comparison</code> and <code>Strategy_theorems</code> cover labelled steps, conversions, finite and infinite reductions, normal forms and strategy properties.
Theorems 2 and 3 from Random Descent	Achieved for the definitions used	<code>olcom_imp_better</code> and <code>owcr_imp_random_descent</code> are proved without admitted lemmas.
Instantiate Example 7	Achieved	The list ARS, inversion strategy, self-critical pairs and mixed critical pairs are formalised in <code>Main_Result</code> .
Quadratic adjacent-swap lower bound	Achieved	The final lower-bound theorem proves $\binom{n}{2} \leq rs $ on descending inputs.
Formal comparison with non-adjacent $O(n \log n)$ algorithms	Not in final scope	The distinction is explained using standard algorithm analysis, but merge sort or heap sort is not formalised in this development.

Objective	Status	Evidence
Bounded-distance swap extension	Not achieved	Retained as future work because it requires a new cost model and a separate family of lower-bound witnesses.

The primary mathematical objective was therefore met, but in a more precise form than the proposal’s phrase “all adjacent-swap algorithms have the same quadratic complexity.” The completed development justifies two statements: all completed inversion-only paths from a fixed input have the same swap count, and every arbitrary adjacent-swap sorting procedure has a quadratic worst-case lower bound. This correction is a strength rather than a reduction of the result.

7.2 Verification Evidence

The main evidence is the dependency-closed Isabelle source. No admitted proof command or additional axiom is used. More importantly, the final theorem depends on a visible chain of intermediate results rather than on one large automation call:

1. normal forms of both list systems are proved equivalent to sortedness
2. the inversion subset is proved to be a strategy
3. all self-critical pairs establish OWCR and hence random descent
4. all mixed critical pairs establish OLCOM and hence global comparison
5. a natural-valued inversion measure constructs finite maximal strategy reductions
6. the measure is calculated on the descending family and
7. the generic comparison transfers the exact strategy cost into the lower bound.

7.3 Representative Examples

Table 7.2 illustrates the exact values involved. The inversion count treats equal elements correctly and the descending distinct lists attain the quadratic bound.

Table 7.2: Representative inversion-only reductions.

Input	Inversions	Consequence
$[]$	0	Already a normal form; the maximal path is empty.
$[1, 1, 2]$	0	Equal values do not form strict inversions.
$[2, 1, 1]$	2	The value 2 crosses both copies of 1; every completed inversion path has two swaps.
$[3, 2, 1]$	3	Two different choices of the first inversion both reach $[1, 2, 3]$ after three swaps.
$[3, 2, 1, 0]$	6	This is $\text{rev}[0.. < 4]$ and $\binom{4}{2} = 6$.

For $[3, 2, 1]$, the nontrivial local peak is visible in the two paths

$$[3, 2, 1] \rightarrow [2, 3, 1] \rightarrow [2, 1, 3] \rightarrow [1, 2, 3]$$

and

$$[3, 2, 1] \rightarrow [3, 1, 2] \rightarrow [1, 3, 2] \rightarrow [1, 2, 3].$$

Each path contains three inversion swaps. The formal overlap diagram is more local than this complete example: after either first step, the branches are joined using two further inversion steps.

An ambient detour demonstrates why the larger system has only a lower-bound comparison. The list $[1, 3, 2]$ has one inversion and can be sorted directly:

$$[1, 3, 2] \rightarrow [1, 2, 3].$$

However, because its source is unsorted, the ambient relation also permits swapping the already ordered pair 1, 3:

$$[1, 3, 2] \rightarrow [3, 1, 2] \rightarrow [1, 3, 2] \rightarrow [1, 2, 3].$$

This path has length three and revisits its source. The example confirms the formal orientation

$$\text{length}(\text{inversion path}) \leq \text{length}(\text{ambient path}).$$

7.4 Discussion of the Results

The direct proof is the shorter way to establish the exact swap count for inversion-only sorting. If the sole objective were the numerical bound, and not developing any of the rewriting framework which was essentially necessary for this project.

The formalisation also exposes why a schematic critical-pair argument expands substantially in a proof assistant. Paper notation silently factors out the unchanged prefix and suffix and treats distant swaps as obviously independent. Isabelle needs lemmas which decompose a list at a valid index, preserve the other pair after a swap, commute disjoint indices and implement the braid relation for overlapping indices.

The final theorem is exact rather than asymptotic. For the chosen input family the lower bound is the concrete value n choose 2. The $\Omega(n^2)$ statement is a corollary at the level of algorithm analysis.

Finally, the result doesn't say "bubble sort and every imaginable adjacent algorithm take exactly the same time." Inversion-only methods use the same number of swaps on a fixed list, but can differ in comparisons and control overhead. Algorithms allowed to make arbitrary adjacent swaps can take more swaps or loop.

Chapter 8

Limitations and Future Work

8.1 Limitations

The first limitation is the cost model. The project counts swaps, not complete execution time. A real implementation must locate a pair, compare values, update storage and maintain loop or recursion state, and this formalisation doesn't deal with these kinds of considerations.

Second, the proposal's comparison with non-adjacent algorithms is not done.

Third, the two inversion count definitions and reduction representations duplicate some reasoning. A reusable library should prove a correspondence theorem or make the direct theory an interpretation of the generic one.

Finally, the generic strategy framework follows the definitions required for the selected paper closely. It has not been packaged as an Archive of Formal Proofs entry, compared systematically with existing Isabelle rewriting libraries, or tested across many unrelated examples. Its generality is formal, but its usability beyond this case is dubious.

8.2 Future Work

The most immediate extension is to turn the theories into a self-contained Isabelle session. Canonical filenames, a ROOT file, release metadata and document antiquotations would allow one command to check both the proofs and the report excerpts.

Chapter 9

Conclusion and Reflection

This project formalised the quadratic obstruction behind sorting with adjacent exchanges. The introductory development proved that every completed sequence of adjacent inversion swaps has length equal to the input’s inversion count and at most $\binom{n}{2}$. The larger development formalised abstract reduction systems, strategies, ordered local commutation and random descent; proved the generic implications needed from [7]; discharged the self and mixed critical pairs for list swaps; and obtained the final theorem that every ambient adjacent swap reduction sorting $\text{rev}[0.. < n]$ has at least $\binom{n}{2}$ steps.

The project changed substantially from its proposal. The original description treated “all adjacent-swap algorithms take the same number of swaps” and “all adjacent-swap algorithms have a quadratic lower bound” almost as the same statement. Formalisation forced a clearer distinction.

The final result is narrower than a full verification of sorting complexity, but it’s precise. It proves an exact finite lower bound under explicit assumptions and explains how a local rewriting argument gives a global algorithmic consequence. The direct and abstract proofs together meet the educational aim of moving from a simple Isabelle argument to a substantial formalisation of a research-paper example.

Bibliography

- [1] F. Baader and T. Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [2] BCS, The Chartered Institute for IT. *BCS Code of Conduct*. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>, accessed 17 August 2026.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, third edition, 2009.
- [4] T. Nipkow. *Programming and Proving in Isabelle/HOL*. Isabelle documentation, 2025. <https://isabelle.in.tum.de/doc/prog-prove.pdf>.
- [5] T. Nipkow, J. C. Blanchette, M. Eberl, A. Gómez-Londoño, P. Lammich, C. Sternagel, S. Wimmer, and B. Zhan. *Functional Algorithms, Verified!* Web publication, 2021. <https://functional-algorithms-verified.org/>.
- [6] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science 2283. Springer, 2002.
- [7] V. van Oostrom. Random Descent. In F. Baader, editor, *Term Rewriting and Applications, RTA 2007*, Lecture Notes in Computer Science 4533, pages 314–328. Springer, 2007. https://doi.org/10.1007/978-3-540-73449-9_24.